

A Large-Scale Empirical Study of Quality Assurance Practices and Gaps in AI Agents

Wuyang Dai

York University

ddai2002@my.yorku.ca

Moses Openja

Polytechnique Montréal

openjamosesopm@gmail.com

Jiho Shin

York University

shinjiho@yorku.ca

Hung Viet Pham

York University

hvhpham@yorku.ca

Song Wang

York University

wangsong@yorku.ca

Abstract—Large language model (LLM)-based agents are increasingly used in software engineering, web automation, research assistance, personal productivity, and other domains. By integrating LLMs with planning, memory, retrieval, tool use, code execution, and interaction with the external environment, these agents can perform complex tasks with increasing autonomy. However, this autonomy also introduces new reliability, safety, and security risks, creating a need for systematic quality assurance (QA) practices tailored to agentic systems.

In this paper, we present a large-scale empirical study of QA practices in 157 open-source LLM-based agent projects, each with at least 100 GitHub stars. We analyze repository documentation, source code, configuration files, tests, and structured review evidence to characterize how current projects address QA practices across four layers: execution surfaces, safeguards, testing artifacts, risk scenarios, and recurring QA gaps. Our results show that existing QA practices primarily focus on validating basic functionality and constraining high-risk actions. However, coverage remains fragmented across the four layers: the breadth of execution surfaces makes QA coverage difficult to trace, controls are not always applied consistently across equivalent routes, tests rarely target boundary, adversarial, or multi-step tool-use failures, and potential risk scenarios are seldom translated into end-to-end QA checks.

Overall, our findings show that current agent QA practices provide only partial protection against high-risk agent actions and do not consistently ensure that model-influenced behavior is bounded across all execution surfaces. We argue that agent QA should move beyond feature-level checks toward end-to-end validation of whether agent workflows remain within intended boundaries when they combine untrusted inputs, model decisions, tools, persistent state, and external APIs.

Index Terms—LLM agent, empirical study, software quality assurance

I. INTRODUCTION

Large language model (LLM)-based agents are rapidly changing how software systems are designed, developed, and used [1], [2]. Unlike traditional LLM applications that primarily return direct responses, agents combine models with planning, memory, tool use, code execution, retrieval, browser interaction, file manipulation, and external services [3]–[6]. These capabilities support multi-step autonomy across software engineering [7], [8], browser and computer automation [9], research assistance [1], [2], and security-oriented evaluation [10], [11].

This shift creates a quality-assurance challenge beyond response correctness. A conventional LLM application may produce an incorrect answer, a hallucinated explanation, or a

misleading recommendation. An agent may instead execute shell commands, modify files, browse websites, call APIs, install packages, or delegate tasks to subagents. Model outputs can therefore influence external actions [9]–[11], with consequences such as corrupted files, privacy leakage, unsafe commands, insecure dependency installation, financial loss, compromised credentials, or harmful third-party interactions [12]–[15]. Agent QA must therefore ask whether model-influenced actions are bounded across interaction and execution surfaces.

Despite the rapid evolution of LLM-based agents and the growing body of work on agent architectures, benchmarks, and security, little is known about how quality assurance is actually engineered in the wild. It remains unclear which QA practices developers adopt, how safeguards and testing are integrated across different execution paths, and whether current repositories provide evidence of end-to-end assurance for model-mediated actions. Open-source repositories provide a unique opportunity to study these engineering practices because they expose testing infrastructure, safeguard implementations, configuration files, documentation, evaluation scripts, and deployment artifacts that collectively reflect observable QA activities during development. To fill this gap, we conduct a large-scale empirical study of 157 open-source LLM-based agent projects, systematically characterizing repository-visible QA practices across architecture, execution surfaces, safeguards, testing artifacts, and risk scenarios. The corpus spans SWE automation (26.8%), personal and multichannel assistants (19.7%), coding assistants (19.1%), research and information agents (13.4%), browser/computer-use automation (12.1%), and domain-specific agents (8.9%).

Because LLM-based agent behavior emerges from interactions among reasoning, planning, execution interfaces, external tools, and runtime safeguards, QA cannot be evaluated solely by inspecting the model outputs. Instead, it must consider how actions are introduced into architecture, exposed through execution surfaces, constrained by safeguards, validated through testing, and connected through potential risk scenarios. We therefore analyze repository-visible QA practices across four layers: *execution surfaces*, *safeguards*, *testing artifacts*, and *risk scenarios*. For each layer, we code what the repository makes inspectable, including action routes, controls, tests, and risk chains. These codes indicate visible QA practices in the repository; they do not show that a mechanism is effective

after deployment. Section III defines the coding rubric and review process.

Through this analysis, we identify recurring QA practices and uncover important QA gaps across the model-to-action workflow. Our results show that agent projects often expose broad execution surfaces and include controls such as approval, sandboxing, permission checks, redaction, and domain scoping. However, these controls are not always applied consistently across equivalent routes. Agent-specific tests for boundary, adversarial, or multi-step tool-use failures remain rare. The risk scenarios we report should therefore be read as candidate model-to-action situations that need boundary review, not as confirmed vulnerabilities. These findings suggest that QA for agents should extend beyond model-output correctness to systematic reasoning about the complete model-to-action workflow. In addition to asking whether the model gives the right answer, agent QA should ask whether model-influenced actions remain authorized, scoped, monitored, and tested when they pass through tools, plugins, browser state, permissions, runtime execution, and user-facing paths.

The remainder of this paper is organized as follows. Section II introduces the background and related work. Section III presents the methodology and empirical study design. Section IV reports the results. Section V discusses threats to validity, and Section VI concludes the paper.

Data Availability: <https://anonymous.4open.science/r/Quality-Assurance-Practices-and-Gaps-in-AI-Agents-85F2/>

II. BACKGROUND AND RELATED WORK

LLM-based agents combine model inference with memory, planning, tools, perception, and cooperation [1], [2]. Prior work has introduced core patterns such as ReAct-style reasoning and acting [3], MRKL-style tool routing [4], reflection and search-based agents [5], [16], [17], memory-centered agents [18], [19], and multi-agent coordination frameworks [6]–[8], [20]. These architectures are usually discussed as capability mechanisms, but they also define trust boundaries: ReAct loops repeatedly mix observations with actions, tool routers map model decisions to external functions, memory systems persist prior context, and multi-agent systems delegate work across roles or workers. We use this literature to code repository architecture, but our focus is narrower: how these architectures expose and constrain safety- and security-relevant execution paths in real projects.

Prior security work motivates this focus. Prompt-injection studies show that untrusted content can redirect model behavior, especially when webpages, documents, emails, issues, or repository files enter agent context [12]–[14]. This risk becomes more concrete in agents because injected instructions can be followed by tool calls, file access, browser actions, or API requests. Agent benchmarks such as ToolEmu, AgentDojo, and Agent Security Bench evaluate attacks and defenses in tool-using environments [9]–[11], while OWASP’s LLM guidance highlights prompt injection, excessive agency, insecure output handling, sensitive information disclosure, and supply-chain risk [15]. Recent MCP and secure-agent studies

further examine MCP security, capability-oriented defenses, and cross-tool state leakage or pollution [21]–[24]. Together, these works identify important risk classes, but they do not directly measure how often open-source agent repositories expose the corresponding tools, policies, tests, and configuration paths.

Empirical OSS security research mines repositories, packages, and static-analysis warnings to understand security practice in the wild [25], [26]; recent agent-testing work studies developer-written test functions across agent frameworks and applications [27]. Our study complements this body of work by providing a repository-level perspective on how QA is engineered in practice. Whereas prior work primarily evaluates agent capabilities, benchmark performance, or individual attacks, our study systematically analyzes repository-visible engineering artifacts that provide evidence of QA in open-source LLM-based agents. We take a broader view in which tests are one evidence source alongside code, configuration, documentation, safeguards, execution surfaces, and structured risk observations. This distinction matters because a repository may document a safeguard without testing it, test a permission component without applying it to every interface, or expose a risky surface without containing a confirmed vulnerability.

III. EMPIRICAL STUDY DESIGN

Figure 1 illustrates the overall workflow of our study, which consists of three stages: repository selection, repository review and annotation, and cross-repository synthesis. During the review stage, we apply a four-layer coding rubric, developed from the literature reviewed in Section II and our initial exploration of selected repositories, to annotate repository artifacts as execution-surface, safeguard, testing-artifact, and risk-scenario records. These layers separate where agents can act, how actions are controlled, how behavior is tested, and which situations require boundary review. During the synthesis stage, we aggregate these coded records to derive corpus-level statistics, QA practice families, and undercover QA problems.

A. Terminology

To avoid ambiguity, we use three terms consistently. *Repository artifacts* are the inspected sources, including documentation, source code, configuration files, manifests, tests, examples, and deployment artifacts. *Coded records* are the structured annotations derived from those artifacts, such as execution-surface records, safeguard records, testing-artifact records, and risk-scenario records. *Analysis layers* are the four categories used to organize those records: execution surfaces, safeguards, testing artifacts, and risk scenarios.

B. Repository Selection

For our analysis, we target popular LLM-based agent applications whose repositories provide enough artifacts to examine QA-relevant behavior and contain executable entry points or package manifests. We focus on public GitHub projects with at least 100 stars as of May 2026, so that the corpus reflects

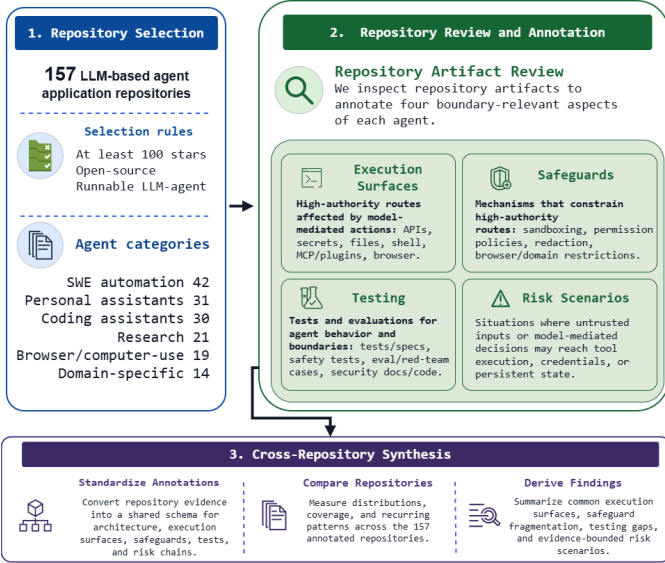


Fig. 1: Workflow of our empirical study.

TABLE I: Corpus composition by project category.

Category	Repos.	Share
SWE automation agents	42	26.8%
Personal / multichannel assistants	31	19.7%
Coding assistants	30	19.1%
Research / information agents	21	13.4%
Browser / computer-use automation	19	12.1%
Domain-specific agents	14	8.9%
Total	157	-

actively visible projects with enough adoption and repository artifacts for systematic review. We first collected candidate repositories using agent-focused search terms, including “AI agent”, “LLM agent”, “autonomous agent”, “coding agent”, “browser agent”, “computer use agent”, and “MCP agent”, supplemented by curated project lists and snowballing from related repositories. A rerun of these queries on 2026-05-21 returned 5,852 summed raw hits before overlap removal and eligibility filtering. We then excluded repositories that did not satisfy three inclusion criteria: the project must integrate an LLM or model-compatible API, contain an executable entry point or package manifest, and expose at least one model-mediated action surface, such as tool calling, shell/process execution, file editing, browser automation, MCP/plugin integration, workflow execution, or persistent agent state. After applying these criteria, our final corpus contains 157 open-source LLM-based agent projects cloned from GitHub. The filtered manifest records each repository’s name, URL, local directory, analyzed commit, timestamps, project-purpose category, interface form, and star count. We treat the corpus as a fixed sample of popular agent applications, not an exhaustive census of all GitHub agent projects. Table I reports the distribution of agent categories.

TABLE II: QA-relevant analysis layers used in the study.

Layer	Description	References
<i>Execution Surfaces</i>	Where an agent can be invoked or can act, including UI, CLI, API, browser, filesystem, shell, plugin, and subagent paths.	[9], [10], [21], [28], [31]
<i>Safeguards</i>	Mechanisms that constrain, validate, or monitor model-mediated actions, such as approval, policy, sandboxing, redaction, and domain scoping.	[9], [12], [14], [15], [23], [29]
<i>Testing Artifacts</i>	Tests, evaluations, benchmarks, reproduction scripts, security tests, and failure-oriented checks.	[9]–[11], [27], [30]
<i>Risk Scenarios</i>	Situations where untrusted context or agent behavior may reach sensitive actions, assets, or persistent state.	[9], [11], [12], [14], [15], [22], [24], [29]

C. Repository Review and Annotation

To systematically characterize repository-visible QA evidence, we first present the coding rubric and then describe the annotation procedure.

1) *Coding Rubric*: The coding rubric consists of four analysis layers: *execution surfaces*, *safeguards*, *testing artifacts*, and *risk scenarios*. We use the rubric to turn repository artifacts into coded records that answer four operational questions. Execution-surface records identify the interfaces and action channels through which an agent can receive requests or affect external systems, following work on tool use and side-effecting agent environments [9]–[11], [28]. Safeguard records identify controls that authorize, scope, monitor, or block agent actions, following prompt-injection and agent-security work on tool, browser, and policy boundaries [9], [12], [14], [15], [29]. Testing-artifact records identify tests, evaluations, benchmarks, and failure-oriented checks for agent behavior under tasks, tools, and environments [9]–[11], [30]. Risk-scenario records identify situations where agent behavior may reach sensitive actions, assets, or persistent state and therefore require boundary review [9], [11], [12], [14], [15], [22], [24], [29]; they are not confirmed vulnerabilities.

2) *Annotation Process*: Before applying the coding rubric, reviewers first familiarized themselves with each repository by reading README files, project documentation, configuration files, and directory structure to understand the project purpose, agent workflow, supported tools, deployment model, and permission assumptions. Reviewers then inspected repository artifacts related to agent execution, including documentation, source code, configuration files, manifests, tests, examples, deployment artifacts, and CI workflows. These artifacts were then coded using the four-layer rubric, recording project purpose, execution surfaces, safeguards, testing artifacts, and risk-scenario candidates.

Our annotation followed three guidelines informed by qualitative-analysis practices in empirical software engineering [32], [33]. First, every label had to point to a repository-visible artifact. Second, capability and QA labels were allowed to co-exist but captured different properties of the artifact: shell, browser, MCP, or plugin access was coded as a capability, while approval prompts, permission checks, sandboxing,

redaction, monitoring, or tests were coded as QA mechanisms associated with that capability. Third, QA labels record the presence of reviewable mechanisms or testing intent, not validated runtime effectiveness; for example, a sandbox setting or test path shows that the repository contains a relevant QA mechanism, but we do not claim that the mechanism prevents all corresponding failures in deployed use.

We developed the label set through open coding and annotation cards. Reviewers first inspected a diverse subset of repositories and wrote cards for recurring behaviors, recording the repository, artifact/path, behavior summary, tentative layer, and uncertainty note. Related cards were grouped and refined into the final labels; for example, a plugin artifact could be an execution-surface record when it added a model-callable extension channel, a safeguard record when it implemented permission checks, or a risk-scenario record when it linked plugin authority to untrusted context.

D. Research Questions

We organize the study around two research questions. RQ1 examines coded records across four analysis layers, and RQ2 identifies the QA problems that remain undercovered within and across those layers.

- **RQ1: What visible QA practice appears in open-source LLM-based agent projects?** To answer this question, we examine four QA layers:
 - **RQ1.1: QA Challenges and Practices at the Execution surfaces.** How do projects expose, separate, document, or qualify the entry points and action channels through which agents can affect external systems?
 - **RQ1.2: QA Challenges and Practices at the Safeguards.** What safeguards are visible for constraining, validating, or monitoring model-mediated behavior before it reaches sensitive actions?
 - **RQ1.3: QA Challenges and Practices at the Testing Layer.** What testing and evaluation artifacts are visible through tests, benchmarks, reproduction paths, robustness checks, security tests, or failure-oriented paths?
 - **RQ1.4: QA Challenges and Practices in Risk scenarios.** What recurring situations allow agent behavior to reach sensitive actions, assets, or persistent state?
- **RQ2: What QA problems remain undercovered in current open-source LLM-based agent projects?** This question identifies missing, fragmented, or weak QA coverage across the four layers, including gaps in end-to-end workflow validation, cross-interface safeguard consistency, robustness and security testing, environment-aware validation, and documentation of bounded actions and trusted assumptions.

IV. RESULTS

This section reports an evidence-led synthesis for the two research questions.

A. RQ1.1: QA Challenges and Practices at the Execution Surface Layer

The execution surface is where an agent can affect external systems. The QA for this layer is therefore not only whether a surface exists, but whether the project can keep those surfaces safe. We derived the QA challenges and QA-practice families through manual review of repository-visible model-to-action paths, configuration paths, tool interfaces, and controls attached to those paths. Table III summarizes the main types of QA challenges and QA practices projects use to address them; percentages in the third column use the QA-challenge project count as the denominator.

1) QA Challenges:

- **Inconsistent control across overlapping action paths** occurs when a project exposes several routes to similar authority, so a control on one route may not cover another route. For example, `google-gemini/gemini-cli` exposes CLI, shell, filesystem, browser, web, and MCP paths, which makes cross-path consistency a central QA challenge.
- **Outbound network/API effects beyond the project boundary** occur when model-mediated behavior can send data or trigger remote actions outside the repository. For example, `CherryHQ/cherry-studio` exposes outbound provider, MCP, search, browser, and local API channels.
- **Credential leakage through configuration** occurs when provider keys, OAuth stores, browser profiles, or MCP environment variables become reachable by tools or subprocesses. For example, `google-gemini/gemini-cli` supports configured MCP environments and provider settings that have to be kept away from logs and tool outputs.
- **Workspace escape through file access** occurs when file tools, downloads, skills, or hooks can read or write outside the intended workspace. For example, `OpenHands/OpenHands` has workspace, sandbox, plugin, hook, and persisted-setting paths around file access.
- **Host process and environment modification through commands** occurs when model-selected commands can change host process state, environment variables, or workspace contents. For example, `openai/codex` exposes shell execution paths whose authority depends on approval and sandbox mode.
- **Extension-driven authority expansion** occurs when MCP servers, plugins, or skills add new model-callable tools and local subprocesses. For example, `aaif-geese/geese` treats MCP extensions as a primary capability.
- **Unreviewed runtime code execution** occurs when configured tools, imported modules, JavaScript, Python snippets, or runtime-loaded plugins execute logic outside the fixed application path. For example, `google-gemini/gemini-cli` supports extension and MCP configuration that can alter the set of callable runtime capabilities.

TABLE III: Execution-surface QA challenges and QA practices.

QA challenge	Projects	QA practice
Inconsistent control across overlapping action paths	152 (96.8%)	Approval and authorization controls: 142 (93.4%); sandboxing and isolation mechanisms: 106 (69.7%); permission and policy controls: 101 (66.4%)
Outbound network/API effects beyond the project boundary	156 (99.4%)	Approval and authorization controls: 143 (91.7%); permission and policy controls: 102 (65.4%)
Credential leakage through configuration	155 (98.7%)	Permission and policy controls: 102 (65.8%); secret management and redaction: 99 (63.9%)
Workspace escape through file access	153 (97.5%)	Sandboxing and isolation mechanisms: 107 (69.9%); permission and policy controls: 101 (66.0%); secret management and redaction: 98 (64.1%)
Host process and environment modification through commands	148 (94.3%)	Approval and authorization controls: 137 (92.6%); sandboxing and isolation mechanisms: 103 (69.6%); permission and policy controls: 98 (66.2%)
Extension-driven authority expansion	127 (80.9%)	Approval and authorization controls: 119 (93.7%); MCP/plugin registration and configuration: 104 (81.9%); permission and policy controls: 97 (76.4%); secret management and redaction: 88 (69.3%)
Unreviewed runtime code execution	123 (78.3%)	MCP/plugin registration and configuration: 93 (75.6%); sandboxing and isolation mechanisms: 89 (72.4%); permission and policy controls: 87 (70.7%)
Browser session and cross-domain exposure	119 (75.8%)	Approval and authorization controls: 110 (92.4%); browser and domain-scoping controls: 87 (73.1%); permission and policy controls: 86 (72.3%); secret management and redaction: 80 (67.2%)

- **Browser session and cross-domain exposure** occurs when browser automation can interact with signed-in sessions, page scripts, cookies, uploads, downloads, or domains. For example, `browser-use/browser-use` exposes browser navigation and page interaction over persistent browser state.

2) QA Practice:

- **Approval and authorization controls** require an explicit grant before a route performs a high-authority action or exposes sensitive API state. For example, `CherryHQ/cherry-studio` protects sensitive local API routes with bearer authentication, while `google-gemini/gemini-cli` uses confirmation and trust modes around interactive actions. This practice addresses inconsistent control across overlapping action paths, outbound network/API effects, host process modification through commands, extension-driven authority expansion, and browser session exposure.
- **Sandboxing and isolation mechanisms** run actions inside a bounded workspace, container, or restricted execution mode so file writes, command effects, and runtime-loaded code are separated from the host. For example, `OpenHands/OpenHands` separates workspace state and sandbox execution, and `openai/codex` routes shell commands through sandbox policy. This practice addresses overlapping action paths, workspace escape through file access, host process modification, and unreviewed runtime code execution.
- **Permission and policy controls** encode rules about which actions, paths, tools, routes, or modes are allowed before execution. For example, `lPanel-dev/MaxKB` uses token permissions and tool filtering around custom Python tools and generated MCP servers. This practice addresses network/API effects, credential leakage, filesystem boundaries, commands, extensions, dynamic code, and browser actions by putting the allowed action set into a reviewable policy layer.
- **Secret management and redaction** limit where credentials can be stored, inherited, logged, or returned through a model/tool path. For example,

`google-gemini/gemini-cli` includes logging redaction and environment sanitization around tool and provider flows. This practice directly addresses credential leakage through configuration and also supports filesystem, plugin, dynamic-code, and browser-session challenges where secrets may be inherited or exposed.

- **MCP/plugin registration and configuration** makes extension points explicit by listing configured servers, tools, commands, or generated plugin interfaces. For example, `aaif-goose/goose` pairs MCP extension support with permission, confirmation, and server-auth mechanisms. This practice addresses extension-driven authority expansion and unreviewed runtime code execution, and it also helps review overlapping action paths when plugin tools duplicate shell, file, browser, or network authority.
- **Browser and domain-scoping controls** restrict which domains, pages, scripts, uploads, downloads, or session data a browser agent can use. For example, `browser-use/browser-use` includes allowed/prohibited domain settings, sensitive-data filtering, IP blocking, storage-state handling, and upload-path gating. This practice addresses browser session and cross-domain exposure, and it also reduces credential leakage and outbound network/API effects when signed-in pages or uploaded files contain sensitive data.

Answer to RQ1.1. QA at the *execution-surface* layer primarily concerns authority boundaries: model decisions can reach credentials, files, commands, plugins, browser state, and external APIs through multiple routes, yet route-level controls are often unevenly applied.

B. RQ1.2: QA Challenges and Practices at the Safeguard Layer

The safeguard layer examines whether projects put clear limits around high-authority agent actions. The main potential issue is incomplete control coverage: a project may protect one route to shell, file, browser, plugin, or secret access while another route to similar authority is less clearly bounded.

Table IV summarizes the main safeguard issues and the safeguard practice families projects use to address them.

1) QA Challenges:

- **Privileged command execution needs approval and containment** because model-selected commands can change host process state, environment variables, or workspace contents. For example, `openai/codex` exposes shell execution paths whose authority depends on approval and sandbox mode.
- **File and workspace access needs boundary control** because file tools, downloads, skills, or hooks can read or write outside the intended workspace. For example, `OpenHands/OpenHands` separates workspace state and sandbox execution, which makes the workspace boundary a central review point.
- **Secrets and credentials need scoped handling** because provider keys, OAuth tokens, browser profiles, and MCP environment variables can become reachable by tools, subprocesses, logs, or provider calls. For example, `google-gemini/gemini-cli` combines provider configuration and MCP environments with logging and redaction paths.
- **Plugin and MCP authority needs extension control** because plugins and MCP servers can add model-callable tools, local subprocesses, and inherited environment access. For example, `aaif-goose/goose` treats MCP/plugin support as a primary capability that expands the set of callable actions.
- **Browser session access needs domain and data scoping** because browser agents can interact with signed-in sessions, page scripts, downloads, uploads, cookies, or domains. For example, `browser-use/browser-use` exposes browser navigation and page interaction over browser state that may persist across tasks or domains.

2) QA Practices:

- **Approval and authorization controls** require an explicit grant before privileged execution or access to sensitive state. For example, `openai/codex` routes shell commands through approval policy, and `aaif-goose/goose` uses confirmation and server-auth mechanisms around extension paths. This practice addresses privileged command execution, plugin/MCP authority, browser session access, and secrets or credentials that should not be exposed without authorization.
- **Sandboxing and isolation mechanisms** run actions inside a bounded workspace, container, or restricted execution mode so command effects, file writes, and runtime-loaded code are separated from the host. For example, `OpenHands/OpenHands` separates workspace state from sandbox execution, while `openai/codex` supports sandbox policy around shell execution. This practice addresses privileged command execution, file/workspace boundary control, secrets and credentials, and plugin or runtime paths that may inherit host authority.

- **Permission and policy controls** encode rules about which commands, files, tools, routes, or modes are allowed before execution. For example, `lPanel-dev/MaxKB` uses token permissions and tool filtering around custom Python tools and generated MCP servers. This practice addresses command execution, file/workspace access, credential handling, plugin/MCP authority, and browser session access by making the allowed action set reviewable.
- **Secret management and redaction** limit where credentials can be stored, inherited, logged, or returned through a model/tool path. For example, `google-gemini/gemini-cli` includes logging redaction and environment sanitization around tool and provider flows. This practice directly addresses secrets and credentials, and it also supports file/workspace, plugin/MCP, command, and browser-session challenges where sensitive values can be inherited or exposed.
- **MCP/plugin registration and configuration** makes extension authority explicit by listing configured servers, tools, commands, or generated plugin interfaces. For example, `aaif-goose/goose` pairs MCP extension support with permission, confirmation, and server-auth mechanisms. This practice addresses plugin/MCP authority and helps review command, file, credential, and browser challenges introduced by extension-provided tools.
- **Browser and domain-scoping controls** restrict which domains, pages, scripts, uploads, downloads, or session data a browser agent can use. For example, `browser-use/browser-use` includes allowed/prohibited domain settings, sensitive-data filtering, IP blocking, storage-state handling, and upload-path gating. This practice addresses browser session access and also reduces credential exposure and file/upload risks when signed-in pages or local files contain sensitive data.

Answer to RQ1.2. For *safeguard* layer, QA practices are widely present, but the key challenge is context-sensitive coverage: the same agent capability may be acceptable in one setting and risky in another, so safeguards need to account for how, where, and under what authority the action is reached.

C. RQ1.3: QA Challenges and Practices at the Testing Layer

Software testing itself is a conventional QA mechanism. From the repository review, we grouped testing artifacts into four testing-practice categories. Across the 157 projects, the counts are: 137 (87.3%) contain conventional tests or specs, 95 (60.5%) contain security/safety-oriented tests, 78 (49.7%) contain evaluation, benchmark, adversarial, or red-team-like paths, and 8 (5.1%) contain prompt-injection, adversarial, jailbreak, or red-team testing paths.

Conventional tests and specs. These tests address ordinary implementation regressions in tool wrappers, API routes, storage, settings, UI helpers, provider clients, and parsers. They are useful because many agent failures still begin as ordinary software bugs: a tool argument is parsed incorrectly, a session

TABLE IV: Safeguard QA challenges and QA practice families.

QA challenge	Projects	QA practice
Privileged command execution needs approval and containment	148 (94.3%)	Approval and authorization controls: 137 (92.6%); sandboxing and isolation mechanisms: 103 (69.6%); permission and policy controls: 98 (66.2%); secret management and redaction: 94 (63.5%)
File and workspace access needs boundary control	153 (97.5%)	Sandboxing and isolation mechanisms: 107 (69.9%); permission and policy controls: 101 (66.0%); secret management and redaction: 98 (64.1%)
Secrets and credentials need scoped handling	155 (98.7%)	Approval and authorization controls: 142 (91.6%); sandboxing and isolation mechanisms: 107 (69.0%); permission and policy controls: 102 (65.8%); secret management and redaction: 99 (63.9%)
Plugin and MCP authority needs extension control	144 (91.7%)	Approval and authorization controls: 135 (93.8%); MCP/plugin registration and configuration: 105 (72.9%); permission and policy controls: 98 (68.1%); secret management and redaction: 93 (64.6%)
Browser session access needs domain and data scoping	119 (75.8%)	Approval and authorization controls: 110 (92.4%); browser and domain-scoping controls: 87 (73.1%); permission and policy controls: 86 (72.3%); secret management and redaction: 80 (67.2%)

setting is not persisted, or a file operation returns the wrong result. For example, `OpenHands/OpenHands` tests sandbox cleanup behavior, including no-sandbox cases, pagination, stale sandbox cleanup, pause failures, invalid cleanup limits, and sorting by creation time; it also tests secret-store initialization, serialization, provider-token handling, and frontend role-permission checks. `browser-use/browser-use` tests CLI sessions, filesystem integration, action-loop detection, coordinate clicking, browser navigation, and setup-command handling.

Security/safety-oriented tests. These tests address boundary regressions around high-authority actions, especially shell execution, filesystem access, browser sessions, domain policy, secrets, approval, sandboxing, and tool permissions. They are more directly agent-specific because the failure condition is not just that a function returns the wrong value, but that an agent action crosses a boundary it should not cross. For example, `browser-use/browser-use` tests that prohibited domains are blocked, allowed-domain lists override broad access, authentication text in a URL does not bypass domain checks, private or localhost IP addresses can be blocked, password fields are removed from DOM snapshots, and sensitive values are filtered from action results. `google-gemini/gemini-cli` tests shell-safety cases such as preferring file-write tools over shell commands for file creation, avoiding destructive shell commands, recovering when sandbox permissions are insufficient, and masking long tool outputs before they are reused.

Evaluation and red-team-like practices. These practices address behavior that only appears after the model chooses actions over one or more steps. They include task evals, benchmark runners, replayable response fixtures, adversarial prompts, and red-team scenarios. For example, `google-gemini/gemini-cli` uses evals for automated tool use, plan mode, subtask delegation, browser-agent workflows, background processes, memory behavior, and eval regression; these checks ask whether the agent chooses the expected tool sequence rather than only whether a helper function works. `Alibaba-NLP/DeepResearch` uses JSONL eval data and evaluation scripts for deep-search and web-agent workflows. `cordum-io/cordum` includes red-team scenarios that exercise approval, policy, tenant isolation, validation, and limit handling.

Prompt-injection and adversarial tests. These tests address the potential issue that untrusted pages, files, prompts, skills, or MCP responses can steer the agent away from the user’s intent or policy. They are a specialized form of security/safety or red-team testing. For example, `Yeachen-Heo/oh-my-claudecode` tests that subagent prompts prepend a fixed system header, preserve the order of system, file, and user context, reject context file paths containing newlines, carriage returns, null bytes, or path traversal, and reject invalid agent-role names. `NousResearch/hermes-agent` includes prompt-injection and red-teaming tests around scheduled tools and bundled skills.

Answer to RQ1.3. Testing remains dominated by conventional regression tests; agent-specific tests for boundary violations, untrusted-context attacks, and multi-step tool behavior are much less systematic.

D. RQ1.4: QA Challenges and Practices in Risk Scenarios Layer

The risk-scenario layer is organized around risk-chain templates. Here, the unit of analysis is the end-to-end path rather than the individual surface or control: behaviors that are ordinary in isolation, such as reading files, invoking tools, reusing state, loading extensions, or using browser sessions, may become risky when connected through model decisions or untrusted context before reaching a sensitive action or asset. Table V reports risk-chain QA challenges and the QA practices used in projects showing each chain.

1) QA Challenges:

- **Secret-to-tool chain.** A stored key, OAuth token, browser profile, or MCP environment variable becomes part of a model-mediated tool path and may then flow into logs, subprocesses, provider calls, or tool outputs. For example, `google-gemini/gemini-cli` combines MCP environments, tool arguments, errors, and provider-facing content, so the review question is whether secret scope is preserved across the whole path.
- **Workspace-to-persistent-state chain.** A model-selected file operation, hook, plugin, or subagent crosses from the task workspace into persisted settings, stored state, or secret storage. For example, `OpenHands/OpenHands` connects workspace state, settings, hooks, plugins, and secret storage,

TABLE V: Risk-chain QA challenges and QA practices.

Risk-chain QA challenge	Projects	QA practices within risk-chain projects
Secret-to-tool chain	149 (94.9%)	Approval and authorization controls: 137 (91.9%); sandboxing and isolation mechanisms: 104 (69.8%); permission and policy controls: 100 (67.1%); secret management and redaction: 96 (64.4%)
Workspace-to-persistent-state chain	128 (81.5%)	Sandboxing and isolation mechanisms: 90 (70.3%); permission and policy controls: 84 (65.6%); secret management and redaction: 82 (64.1%)
Extension-to-authority chain	137 (87.3%)	Approval and authorization controls: 129 (94.2%); MCP/plugin registration and configuration: 101 (73.7%); permission and policy controls: 94 (68.6%); secret management and redaction: 91 (66.4%)
Mode-switch-to-privilege chain	123 (78.3%)	Approval and authorization controls: 114 (92.7%); sandboxing and isolation mechanisms: 84 (68.3%); permission and policy controls: 82 (66.7%)
Runtime-code chain	99 (63.1%)	Approval and authorization controls: 92 (92.9%); MCP/plugin registration and configuration: 75 (75.8%); sandboxing and isolation mechanisms: 73 (73.7%); permission and policy controls: 69 (69.7%)
Model-to-process chain	131 (83.4%)	Approval and authorization controls: 123 (93.9%); sandboxing and isolation mechanisms: 91 (69.5%); permission and policy controls: 91 (69.5%); secret management and redaction: 84 (64.1%)
Session-to-action chain	60 (38.2%)	Approval and authorization controls: 53 (88.3%); browser and domain-scoping controls: 50 (83.3%); permission and policy controls: 44 (73.3%); secret management and redaction: 38 (63.3%)
Untrusted-context-to-action chain	45 (28.7%)	Approval and authorization controls: 39 (86.7%); permission and policy controls: 27 (60.0%); secret management and redaction: 25 (55.6%); browser and domain-scoping controls: 24 (53.3%)

so the review question is whether the intended workspace boundary still holds after state is reused.

- **Extension-to-authority chain.** A configured plugin, MCP server, or skill adds a new model-callable action that inherits local subprocess, network, file, or environment authority. For example, `aaif-goose/goose` treats MCP extensions as first-class capabilities, so the review question is whether newly added tools inherit the same trust and permission assumptions as built-in tools.
- **Mode-switch-to-privilege chain.** An action that is checked in an interactive path is reachable through another mode, such as an automatic mode, setup hook, API route, headless path, or alternate tool. For example, `openai/codex` includes multiple shell-related modes and control paths, so the review question is whether the same privileged action remains bounded after the execution mode changes.
- **Runtime-code chain.** Configuration, imported modules, JavaScript, Python snippets, or runtime-loaded plugins introduce executable logic after the fixed application code has been reviewed. For example, `google-gemini/gemini-cli` combines extension and MCP configuration with runtime capability changes, so the review question is whether loaded code is subject to the same review boundary as built-in code.
- **Model-to-process chain.** A model-selected action reaches a host process, environment variable, background job, or workspace mutation. For example, `openai/codex` exposes command execution through more than one control path, so the review question is whether command authority is bounded at the action level rather than only at one interface.
- **Session-to-action chain.** Browser state, cookies, storage, uploads, downloads, or signed-in pages persist across domains or tasks and later influence an agent action. For example, `browser-use/browser-use` connects navigation, page interaction, storage state, and uploads, so the review question is whether session state remains scoped to the intended task and domain.
- **Untrusted-context-to-action chain.** A page, file, prompt, tool output, or MCP response influences a later tool call,

command, file operation, or data disclosure. For example, `Yeachen-Heo/oh-my-claudecode` handles subagent context files, so the review question is whether untrusted context can move from input material into later privileged actions.

2) QA Practices:

QA practices in this layer can be grouped into two broad categories.

- **Chain-interruption practices** stop a risky path at a critical transition point, such as when a model-mediated workflow reaches a privileged action, host execution, or sensitive data.
 - **Authorize the transition to a privileged action.** Approval and authorization controls interrupt a chain when a model-mediated path reaches a sensitive action. This practice appears in 137 of 149 secret-to-tool chains, 129 of 137 extension-to-authority chains, 114 of 123 mode-switch-to-privilege chains, 123 of 131 model-to-process chains, 53 of 60 session-to-action chains, and 39 of 45 untrusted-context-to-action chains. For example, `openai/codex` uses an approval policy around command execution paths, and `aaif-goose/goose` uses confirmation and server-auth mechanisms around extension paths.
 - **Contain execution or state effects.** Sandboxing and isolation mechanisms interrupt a chain by preventing a tool, command, runtime module, or workspace operation from inheriting unrestricted host authority. This practice appears in 104 of 149 secret-to-tool chains, 90 of 128 workspace-to-persistent-state chains, 84 of 123 mode-switch-to-privilege chains, 73 of 99 runtime-code chains, and 91 of 131 model-to-process chains. For example, `OpenHands/OpenHands` separates workspace state from sandbox execution, while `openai/codex` applies sandbox policy to shell execution.
 - **Stop secret propagation.** Secret management and redaction interrupt a chain by preventing credentials from being inherited, logged, returned through tool

output, or sent into provider-facing content. This practice appears in 96 of 149 secret-to-tool chains, 82 of 128 workspace-to-persistent-state chains, 91 of 137 extension-to-authority chains, 84 of 131 model-to-process chains, 38 of 60 session-to-action chains, and 25 of 45 untrusted-context-to-action chains. For example, `google-gemini/gemini-cli` includes logging redaction and environment sanitization around tool and provider flows.

- *Systematic risk-avoidance practices* reduce the chance that such a path forms in the first place by limiting risky combinations.
 - **Scope allowed paths, tools, modes, and data flows.** Permission and policy controls avoid risky chains by making the allowed action set explicit before execution. This practice appears in 100 of 149 secret-to-tool chains, 84 of 128 workspace-to-persistent-state chains, 94 of 137 extension-to-authority chains, 82 of 123 mode-switch-to-privilege chains, 69 of 99 runtime-code chains, 91 of 131 model-to-process chains, 44 of 60 session-to-action chains, and 27 of 45 untrusted-context-to-action chains. For example, `lpanel-dev/MaxKB` uses token permissions and tool filtering around custom Python tools and generated MCP servers.
 - **Make extension and runtime authority reviewable.** MCP/plugin registration and configuration avoid hidden authority chains by making added tools, servers, commands, or generated interfaces visible before they become model-callable. This practice appears in 101 of 137 extension-to-authority chains and 75 of 99 runtime-code chains. For example, `aaif-goose/goose` exposes MCP extension configuration as a primary control point.
 - **Scope browser sessions and domains.** Browser and domain-scoping controls avoid session-to-action chains by limiting which pages, domains, uploads, downloads, storage state, or signed-in sessions can influence later actions. This practice appears in 50 of 60 session-to-action chains and 24 of 45 untrusted-context-to-action chains. For example, `browser-use/browser-use` includes domain restrictions, sensitive-data filtering, storage-state handling, and upload-path controls.

Answer to RQ1.4. For *risk scenarios* layer, common QA practices include design-time risk reduction and chain interruption at critical points, using mechanisms such as authorization, isolation, and redaction.

E. RQ2: Undercovered QA Problems Across the Four RQ1 Layers

This RQ examines which QA problems remain undercovered across the four RQ1 layers. We treat a problem as undercovered when repositories expose the problem but

lack the most directly relevant QA practice, or when the corresponding practice is implemented only as a local control for a specific route, mode, or artifact type rather than as systematic coverage. Table VI summarizes the largest direct gaps by RQ1 dimension.

Execution surfaces. The table shows that the weakest coverage is not for obscure surfaces, but for common routes such as credentials, network/API calls, file access, shell commands, and dynamic code. In these cases, the project exposes an action route but lacks the direct control expected for that route. For example, `vercel-labs/agent-browser` is counted in the credential gap because its review record includes environment secrets, browser state, injected headers, and cloud profiles reachable from browser/CLI paths, while it does not match the secret-management/redaction practice rule. `0x4m4/hexstrike-ai` is counted in the file and command gaps because its review record links user/model-controlled paths and shell commands to API/MCP routes, while the corresponding sandbox/isolation practice is not present under our rule. The practical implication is that surface inventory alone is insufficient; each action route needs an attached boundary such as redaction, permission policy, or sandboxing.

Safeguards. The safeguard gaps show that broad controls do not always cover the specific action that needs protection. Secret handling, workspace containment, MCP/plugin extension, and browser session control need different direct practices, but projects often rely on more general approval/authentication or configuration assumptions. For example, `multica-ai/multica` is counted in the MCP/plugin gap because its review record describes stored per-agent MCP configuration being passed to Claude Code as a local tool channel, but it lacks the direct MCP/plugin registration practice under our reproducible rule. `0x4m4/hexstrike-ai` is counted in the browser/domain gap because its browser agent can visit arbitrary URLs and return browser-observed secrets, but no browser/domain-scoping control is matched. The remaining QA gap is therefore not simply “add a safeguard”; it is to bind the right safeguard to the specific sensitive action.

Testing artifacts. The testing gaps are the most direct sign that QA practice has not caught up with agent-specific failure modes. Projects may have ordinary tests, but the missing direct practices are adversarial tests for untrusted context, replay/regression tests for multi-step behavior, and security/safety tests for high-authority actions. For example, `browser-use/browser-use` is counted in the untrusted-context testing gap because its review record links model-reachable browser JavaScript execution and browser-domain policy defaults to prompt-injection-style risk, while it does not match the prompt-injection/adversarial test rule. `reworkd/AgentGPT` is similarly counted because external snippets are inserted into summarization prompts, but the matching adversarial test practice is absent. The remaining gap is failure sensitivity: tests should fail when a model-mediated action escapes an approval, sandbox, credential, plugin, file, or browser boundary.

TABLE VI: Undercovered QA problems by RQ1 dimension. Percentages show the share of affected projects lacking the corresponding direct QA practice.

Dimension	Undercovered QA problems	Projects lacking direct QA practice	Remaining QA gap
RQ1.1 Execution surfaces	Credential configuration, network/API routes, file access, command execution, and dynamic-code paths.	Secret management/redaction: 36.1%; permission/policy controls: 34.6%; sandboxing/isolation: 30.1% for file access and 30.4% for command execution.	Projects often expose the routes, but do not always define or enforce the boundary that should apply to each route.
RQ1.2 Safeguards	Secret handling, file/workspace containment, plugin/MCP extension control, and browser/session scoping.	Secret management/redaction: 36.1%; sandboxing/isolation: 30.1%; MCP/plugin registration: 27.1%; browser/domain scoping: 26.9%.	Controls are present, but the direct control for the privileged action is not always present.
RQ1.3 Testing artifacts	Adversarial tests for untrusted context, regression or replay tests for multi-step behavior, and boundary tests for high-authority actions.	Prompt-injection/adversarial tests: 97.8%; evaluation/red-team-like paths: 52.4%; security/safety-oriented tests: 39.7%.	Many tests check components or expected workflows rather than whether model-mediated actions stay inside required boundaries.
RQ1.4 Risk scenarios	Credential-flow, command-execution, file-boundary, extension-authority, and runtime-code risk chains.	Secret management/redaction: 35.6%; sandboxing/isolation: 30.5% for command risk chains and 29.7% for file-boundary risk chains; MCP/plugin registration: 26.3%.	Risk chains can remain general concerns unless projects turn them into bounded-action requirements and checks.

Risk scenarios. The risk-scenario gaps identify chains that need to be converted into QA requirements. These chains include credential flow into tools or subprocesses, commands affecting host state, workspace-boundary crossing, extension-based authority, and runtime-loaded code. For example, `Fosowl/agentSeek` is counted in the credential-flow gap because its review record links LLM-generated Python execution to backend secrets, while secret-management/redaction is not matched. `0x4m4/hexstrike-ai` is counted in the command-execution gap because an unauthenticated API route can reach host shell execution, while sandbox/isolation is not matched. The remaining gap is operationalization: each chain should name the trigger, protected asset, required control, and regression check that would stop the chain.

Recommendations. First, projects should define controls by protected action or asset rather than by interface, so the same secret, file, shell, plugin, browser, and network boundary applies across CLI, API, MCP, browser, headless, and batch routes. Second, each high-authority route or risk chain should have a failure-oriented test or replay case that fails when the required approval, sandbox, policy, redaction, domain scope, or extension check is removed.

Answer to RQ2. Many undercovered QA problems are agent-specific rather than traditional software defects: model-mediated tool use, session reuse, extension authority, and untrusted-context propagation often lack corresponding QA practices beyond ordinary tests, access checks, or configuration review.

V. THREATS

Internal validity. Our analysis is based on repository-level artifacts, including source code, documentation, configuration, tests, and structured review notes. This view captures what developers make inspectable in the repository, but may not include hosted services, provider-side controls, deployment-specific settings, or dependency behavior outside the inspected snapshot. Automated search and LLM-assisted summarization helped identify the candidate artifacts, but may introduce

false positives, false negatives, or inconsistent terminology. We mitigate this risk through manual verification of candidate artifacts and a fixed coding schema for quantitative analysis. Because of objective was to inductively develop a repository-level coding framework through iterative open coding, the coding categories evolved during analysis instead of using a fixed predefined codebook. Consequently, we did not compute inter-rater agreement during taxonomy construction. To improve consistency, reviewers continuously refined label definitions, discussed ambiguous cases, and resolved disagreements through consensus before applying the finalized coding rubric across the corpus. This approach follows common qualitative coding practices used for inductive taxonomy development in empirical software engineering. [34], [35].

Construct validity. The four analysis layers provide an operational coding rubric rather than a universal taxonomy of agent QA. Some projects could reasonably be grouped differently, especially when one artifact plays multiple roles, such as exposing a tool, constraining its use, and documenting a risk scenario. To reduce this ambiguity, we separated capabilities from QA mechanisms and kept structured records for paths, labels, and review notes. Our counts should be read as repository-visible QA practices, not as measurements of runtime effectiveness.

External validity. The corpus is a fixed May 2026 snapshot of popular open-source GitHub projects and overrepresents coding and browser/computer-use agents. It is not a census of all agent repositories; closed-source, enterprise, and small prototype agents may differ. GitHub stars are descriptive metadata, not a security-maturity measure, and repositories may have changed after collection.

VI. CONCLUSION

This paper presented an empirical study of 157 open-source LLM-based agent projects. Using a four-layer coding rubric, we characterized execution surfaces, safeguards, testing artifacts, and risk scenarios from repository artifacts and coded records. The results show that QA practices are common, but their coverage across agent workflows remains uneven. The main gap is not whether projects contain QA artifacts,

but whether those artifacts cover the full agent workflow. Future agent QA should make tool authority explicit, connect safeguards and tests to the situations where sensitive actions are reached, and add failure-oriented checks for boundary, adversarial, and multi-step tool-use cases.

REFERENCES

- [1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, *et al.*, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [2] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, *et al.*, “The rise and potential of large language model based agents: A survey,” *arXiv preprint arXiv:2309.07864*, 2023.
- [3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.
- [4] E. Karpas, O. Abend, Y. Belinkov, B. Lenz, O. Lieber, N. Ratner, Y. Shoham, H. Bata, Y. Levine, K. Leyton-Brown, D. Muhlgay, N. Rozen, E. Schwartz, G. Shachaf, S. Shalev-Shwartz, A. Shashua, and M. Tenenholz, “Mrkl systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning,” *arXiv preprint arXiv:2205.00445*, 2022.
- [5] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, 2023.
- [6] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, *et al.*, “Autogen: Enabling next-gen llm applications via multi-agent conversation framework,” in *Conference on Language Modeling*, 2024.
- [7] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [8] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, “ChatDev: Communicative agents for software development,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (L.-W. Ku, A. Martins, and V. Srikumar, eds.), (Bangkok, Thailand), pp. 15174–15186, Association for Computational Linguistics, Aug. 2024.
- [9] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 82895–82920, 2024.
- [10] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the risks of llm agents with an llm-emulated sandbox,” in *International Conference on Learning Representations*, 2024.
- [11] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, “Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents,” in *International Conference on Learning Representations*, 2025.
- [12] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, AISec ’23*, (New York, NY, USA), p. 79–90, Association for Computing Machinery, 2023.
- [13] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, L. Y. Zhang, and Y. Liu, “Prompt injection attack against llm-integrated applications,” *arXiv preprint arXiv:2306.05499*, 2023.
- [14] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *USENIX Security Symposium*, 2024.
- [15] OWASP Foundation, “Owasp top 10 for large language model applications.” <https://owasp.org/www-project-top-10-for-large-language-model-applications/>, 2025.
- [16] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. L. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in *Advances in Neural Information Processing Systems*, 2023.
- [17] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, “Language agent tree search unifies reasoning acting and planning in language models,” in *International Conference on Machine Learning*, 2024.
- [18] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.
- [19] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *Transactions on Machine Learning Research*, 2024.
- [20] G. Li, H. A. Al Kader Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, “Camel: communicative agents for ”mind” exploration of large language model society,” in *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS ’23*, (Red Hook, NY, USA), Curran Associates Inc., 2023.
- [21] M. M. Hasan, H. Li, E. Fallahzadeh, G. K. Rajbahadur, B. Adams, and A. E. Hassan, “Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers,” *arXiv preprint arXiv:2506.13538*, 2025.
- [22] Y. Yang, C. Gao, D. Wu, Y. Chen, Y. Li, and S. Wang, “Mcpsecbench: A systematic security benchmark and playground for testing model context protocols,” *arXiv preprint arXiv:2508.13220*, 2025.
- [23] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating prompt injections by design,” *arXiv preprint arXiv:2503.18813*, 2025.
- [24] Z. Li, J. Cui, X. Liao, and L. Xing, “Les dissonances: Cross-tool harvesting and polluting in pool-of-tools empowered llm agents,” in *33rd Annual Network and Distributed System Security Symposium, NDSS 2026, San Diego, California, USA, February 24-27, 2026*, (San Diego, CA), February 2026.
- [25] B. Aloraini, M. Nagappan, D. M. German, S. Hayashi, and Y. Higo, “An empirical study of security warnings from static application security testing tools,” *Journal of Systems and Software*, vol. 158, p. 110427, 2019.
- [26] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. S. Maddila, and L. A. Williams, “What are weak links in the npm supply chain?,” in *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, pp. 331–340, 2022.
- [27] M. M. Hasan, H. Li, E. Fallahzadeh, G. K. Rajbahadur, B. Adams, and A. E. Hassan, “An empirical study of testing practices in open source ai agent frameworks and agentic applications,” *Empirical Software Engineering*, vol. 31, no. 5, p. 124, 2026.
- [28] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” *Advances in neural information processing systems*, vol. 36, pp. 68539–68551, 2023.
- [29] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 10471–10506, 2024.
- [30] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, *et al.*, “Agentbench: Evaluating llms as agents,” in *International Conference on Learning Representations*, vol. 2024, pp. 52989–53046, 2024.
- [31] J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: Agent-computer interfaces enable automated software engineering,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 50528–50652, 2024.
- [32] C. B. Seaman, “Qualitative methods in empirical studies of software engineering,” *IEEE Transactions on Software Engineering*, vol. 25, no. 4, pp. 557–572, 1999.
- [33] D. S. Cruzes and T. Dybå, “Recommended steps for thematic synthesis in software engineering,” in *Proceedings of the 2011 International Symposium on Empirical Software Engineering and Measurement*, pp. 275–284, 2011.
- [34] J. Díaz, J. Pérez, C. Gallardo, and Á. González-Prieto, “Applying inter-rater reliability and agreement in collaborative grounded theory studies in software engineering,” *Journal of Systems and Software*, vol. 195, p. 111520, 2023.

[35] M. Openja, F. Khomh, A. Foundjem, Z. M. Jiang, M. Abidi, and A. E. Hassan, "An empirical study of testing machine learning in the wild,"

ACM transactions on software engineering and methodology, vol. 34, no. 1, pp. 1–63, 2024.